

SYSTEMANALYSE UND ENTWURF

CineTicket

Ein Kino-Ticketsystem — von der Anforderungsanalyse über die UML-Modelle bis zum klickbaren Prototyp.

CINETICKET

So ist das Projekt gebaut



Alle Inhalte — Personas, Stories, UML-Klassen, Zustände — liegen als strukturierte JSON-Dateien vor. Die Website rendert ihre Diagramme daraus live als SVG; Inhalt und Darstellung können nicht auseinanderlaufen.

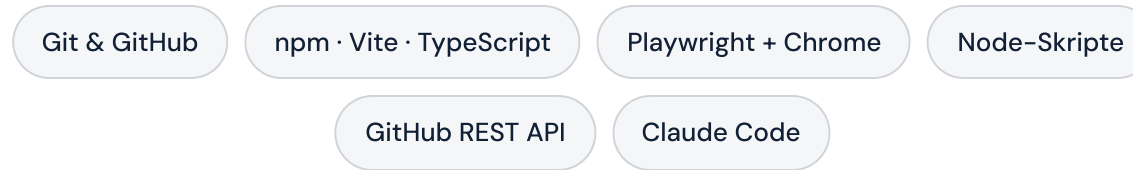
Zwei Detailgrade auf der Website: Einfach ist immer eine echte Teilmenge von Erweitert.

CINETICKET

So ist es entstanden



WERKZEUGE



Jede Änderung folgt demselben Weg: Regeln lesen, gezielt ändern, real testen, festhalten, deployen. Umgesetzt KI-gestützt mit Claude Code — veröffentlicht wird nur, was geprüft ist.

Nichts geht ungeprüft live — Lint, Build und ein echter Browser-Test sichern jeden Stand.

CINETICKET

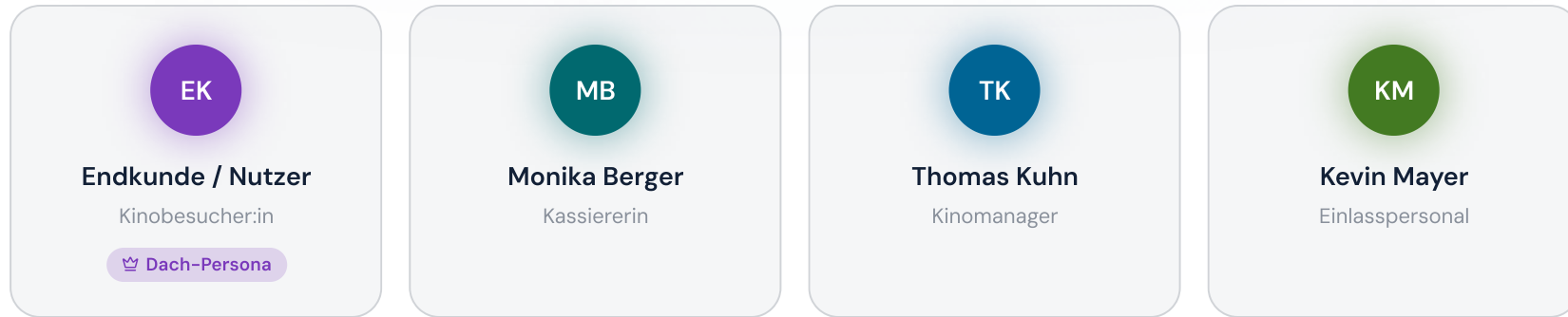
Der Weg: drei Stationen



Aus den Anforderungen entstehen die Modelle, aus den Modellen der Prototyp — jede Station baut auf der vorigen auf. Ideen über den MVP hinaus schließen den Bogen.

PERSONAS

Kundschaft und Betrieb



Eine Rolle draußen, drei drinnen: Der Endkunde bucht. Im Betrieb verkauft Monika an der Kasse, Thomas steuert Programm und Umsatz, Kevin kontrolliert den Einlass. Beide Seiten haben eigene Ziele, daraus entstehen verschiedene Anforderungen.

Jede User Story gehört später zu genau einer Persona.

PERSONAS

So tickt die Kundschaft

**Endkunde / Nutzer**
Kinobesucher:in · 35 Jahre

„In 60 Sekunden gebucht, der Lieblingsplatz reserviert.“

 ZIELE	 FRUSTRATIONEN
● Schnell und unkompliziert Tickets kaufen (mobil unter 60 Sekunden) ● Den besten freien Platz per KI-Vorschlag mit einem Tipp übernehmen ● Rabatte (Senior, Student, Kind, Gruppe) ohne Hürden anwenden ● Barrierefreie Plätze und Begleitperson-Optionen klar erkennen ● Bonuspunkte sammeln, einsehen und einlösen	● Lange Ladezeiten und komplizierte Mobile-Buchung ● Sitzplan ohne Komfort- oder Barrierefreiheits-Details ● Gruppen-Tickets nur einzeln buchbar ● Keine modernen Zahlungsmittel (Apple Pay / Google Pay)

Die Kundenseite am Beispiel Endkunde: schnell mobil buchen, den Lieblingsplatz sichern. Frustrationen wie lange Ladezeiten oder umständliche Gruppenbuchung prägen den Buchungs-Wizard und den Sitz-Hold.

PERSONAS

So tickt der Betrieb

 Monika Berger Kassiererin · 38 Jahre	<i>„Schnell, korrekt, freundlich – in dieser Reihenfolge.“</i>
 ZIELE • Tickets schnell und fehlerfrei verkaufen • Warteschlangen minimieren • Einfache Stornierungen durchführen	 FRUSTRATIONEN • Langsame Ladezeiten im System • Komplizierte Sitzplatzbuchung unter Zeitdruck • Keine Übersicht bei ausverkauften Vorstellungen

Die Betriebsseite am Beispiel Monika: schnell, korrekt und freundlich verkaufen. Ihre Frustration sind langsame Systeme unter Zeitdruck, daraus entsteht der Schnellverkauf mit Tastatur-Shortcuts.

Im Vollausbau werden aus den Rollen konkrete Profile — von der Stammkundin bis zur Familienmutter.

USER STORIES

Ein Satz, drei Antworten

Als **Persona** möchte ich **Ziel** um **Nutzen** .

Persona – wer braucht es?

Ziel – was soll möglich sein?

Nutzen – warum lohnt es sich?

Jede User Story beschreibt eine Anforderung aus Nutzersicht, in einem Satz mit Persona, Ziel und Nutzen. Dazu kommen testbare Akzeptanzkriterien als Definition von „fertig“.

USER STORIES

Die wichtigste Regel: der Sitz-Hold

U47

Hohe Priorität

Release 1 – MVP

 Lara

Als Stammkundin möchte ich, dass mein gewählter Platz beim Checkout verbindlich für eine begrenzte Zeit reserviert wird, damit ihn niemand parallel buchen kann (keine Doppelbuchung).

AKZEPTANZKRITERIEN

- ✓ Platz wird zu Beginn des Checkouts auf Status RESERVIERT gesetzt
- ✓ Hold läuft nach 10 Minuten automatisch ab und gibt den Platz wieder frei
- ✓ Eine parallele Buchung desselben Platzes wird abgewiesen
- ✓ Erst die bezahlte Buchung setzt den Platz auf BELEGT

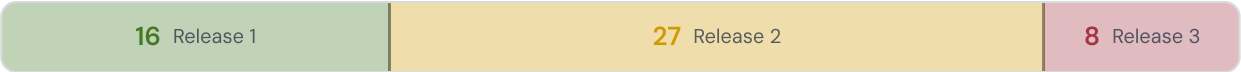
U47 ist der fachliche Kern: Der gewählte Platz wird beim Checkout verbindlich reserviert, das verhindert Doppelbuchungen. Dieselbe Story taucht im Sequenzdiagramm und im Zustandsautomaten wieder auf.

USER STORIES

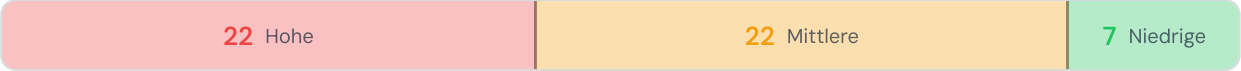
Priorität und Release: der Bauplan

30 Stories im Einfach-Modus c 51 im Erweitert-Modus

NACH RELEASE



NACH PRIORITÄT (MOSCOW)

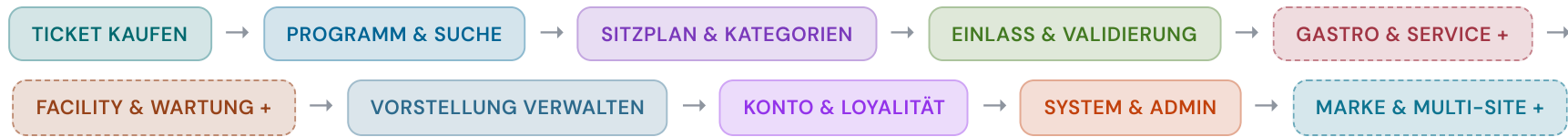


Jede Story trägt eine MoSCoW-Priorität und ein Release. Zusammen ergeben sie den Bauplan: Release 1 ist die schmale, lauffähige Scheibe, der MVP. Spätere Releases erweitern dort, wo es Wert stiftet.

Jede Story kennt ihre Persona, ihre Aktivität und ihr Release — nichts hängt in der Luft.

STORY MAP

Das Backbone: die Nutzerreise



„+“ = Aktivität kommt erst im Erweitert-Modus dazu (7 → 10)

Die Story Map ordnet alle Stories zweidimensional: waagrecht die Nutzerreise, senkrecht die Auslieferung in Releases. Das Rückgrat bilden die Aktivitäten — vom Ticketkauf über Sitzplan und Einlass bis zur Verwaltung.

STORY MAP

Waagerecht schneiden: Releases

ALLE AKTIVITÄTEN →

Release 1 – MVP die lauffähige Scheibe, die der Prototyp zeigt

16 Stories

Release 2 – Erweiterung Komfort, Gastro & Service, Facility

27 Stories

Release 3 – Vollausbau Empfehlungen, Reports, Multi-Site

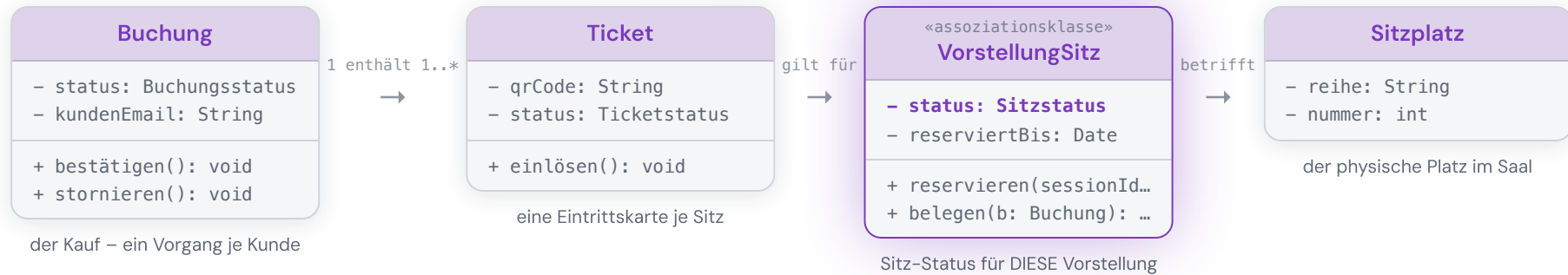
8 Stories

Release 1 ist die schmale, lauffähige Scheibe quer durch alle Aktivitäten. Die weiteren Releases erweitern das System dort, wo es Wert stiftet, statt einfach „mehr Features“.

Auf der Website ist die Karte interaktiv — jedes Release-Band lässt sich hervorheben, jede Karte öffnet ihre Story.

KLASSENDIAGRAMM

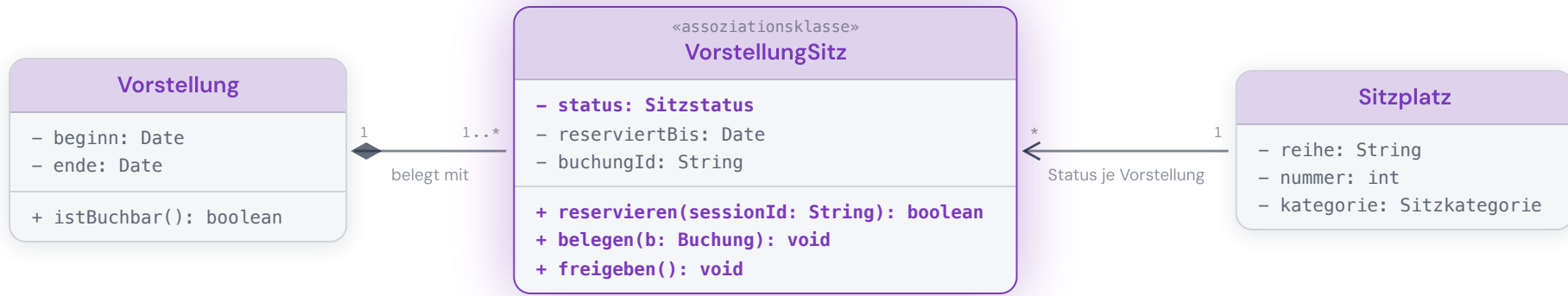
Das Herzstück: eine Buchung



Vier Klassen, vier klare Aufgaben: Eine Buchung ist der Kauf. Sie enthält für jeden Platz ein Ticket. Jedes Ticket gilt für einen VorstellungSitz, den Status eines Sitzes in genau dieser Vorstellung. Der physische Sitzplatz im Saal bleibt davon unberührt.

KLASSENDIAGRAMM

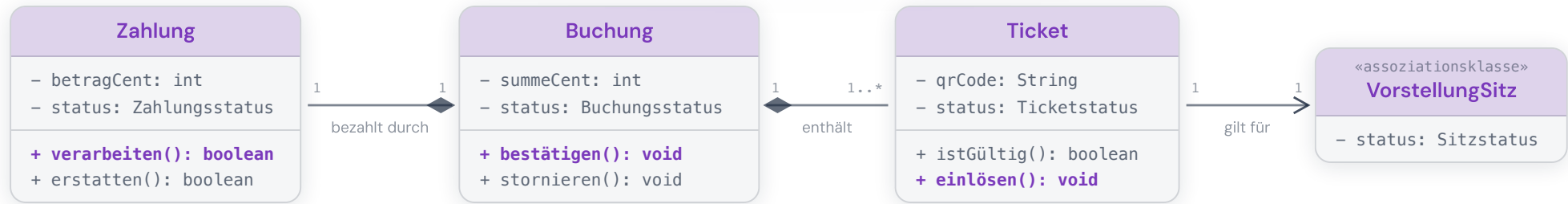
Der Schlüssel: VorstellungSitz



Deshalb braucht es die Assoziationsklasse: Ein Sitzplatz existiert dauerhaft, sein Status wechselt aber je Vorstellung. `reservieren()`, `belegen()` und `freigeben()` setzen den Hold durch und verhindern Doppelbuchungen, für genau diese eine Vorstellung.

KLASSENDIAGRAMM

Wer macht was: Objekt vs. Service

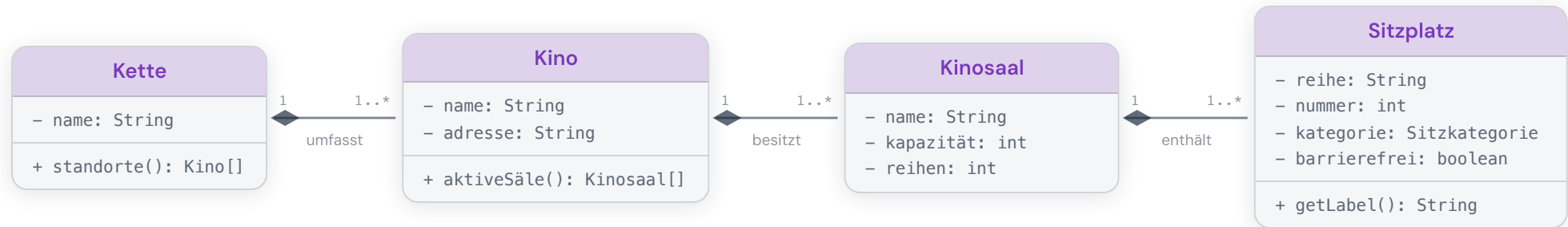


Statusändernde Operationen sitzen am Objekt, dessen Zustand sie ändern: `Zahlung.verarbeiten()`, `Buchung.bestätigen()`, `Ticket.einlösen()`. Buchung ist nicht der BuchungService.

«control» steuert, «entity» weiß — der BuchungService orchestriert nur und hält selbst keine Daten.

KLASSENDIAGRAMM

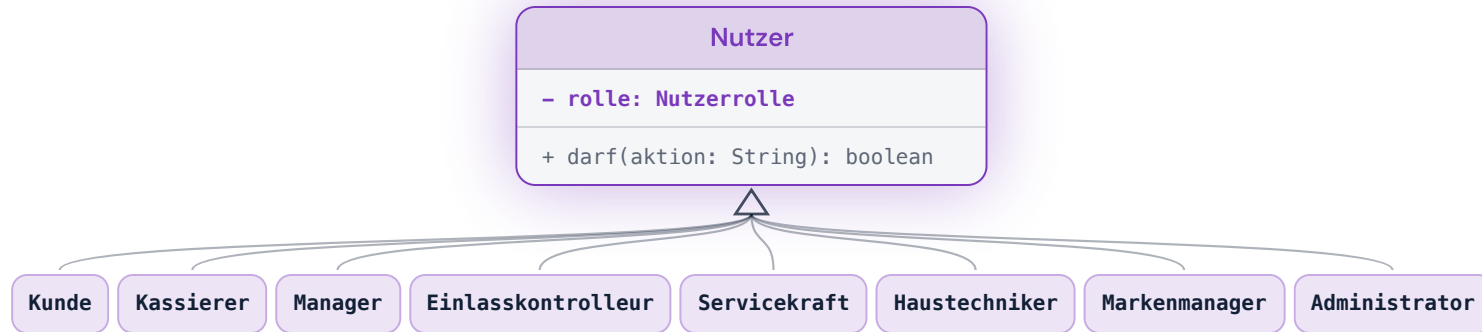
Vom Saal zum einzelnen Sitz



Woher der Sitzplatz kommt: eine Kompositions-Kette (◆). Eine Kette umfasst Kinos, ein Kino besitzt Säle, ein Saal enthält Sitzplätze. Der Teil existiert nicht ohne das Ganze.

KLASSENDIAGRAMM

Rollen erben von Nutzer



Vom Kunden bis zum Administrator erben alle Akteure von Nutzer. Was jemand darf, entscheidet nicht die Klasse, sondern das Enum `Nutzerrolle`, geprüft über `darf(aktion)`.

SEQUENZDIAGRAMME

Der Buchungsablauf über die Zeit

Online-Buchung (Kunde)

Vorstellung → Sitzplan → Hold → Zahlung → QR-Ticket

Kassenverkauf / POS (Kassiererin)

Tickets in den Warenkorb, an der Kasse bezahlen

Storno & Rückerstattung

Buchung stornieren: Ticket ungültig, Zahlung erstattet, Sitz frei

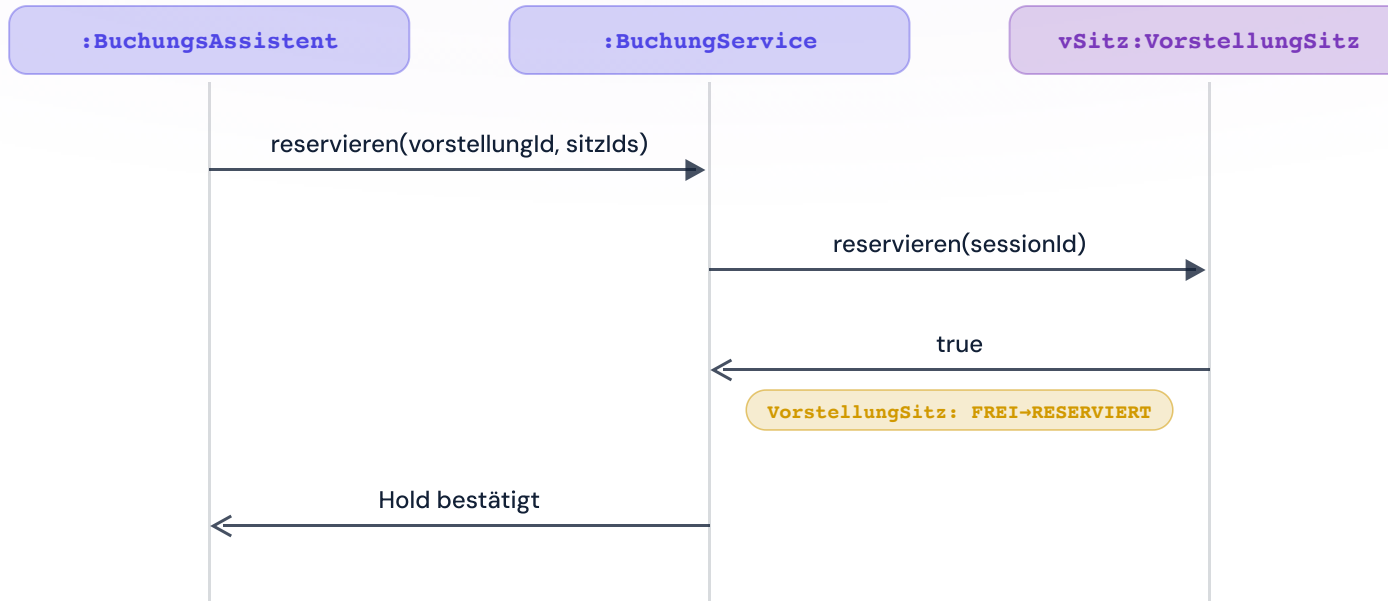
Einlass & Validierung

QR-Code scannen, gültiges Ticket einlösen

Ein Sequenzdiagramm zeigt, wer wann welche Nachricht an wen schickt. Statt eines überladenen Diagramms gibt es fokussierte Abläufe: Online-Buchung, Kasse, Storno, Einlass. Die nächsten Folien zoomen in die Online-Buchung.

SEQUENZDIAGRAMME

Der Hold: 10 Minuten verbindlich

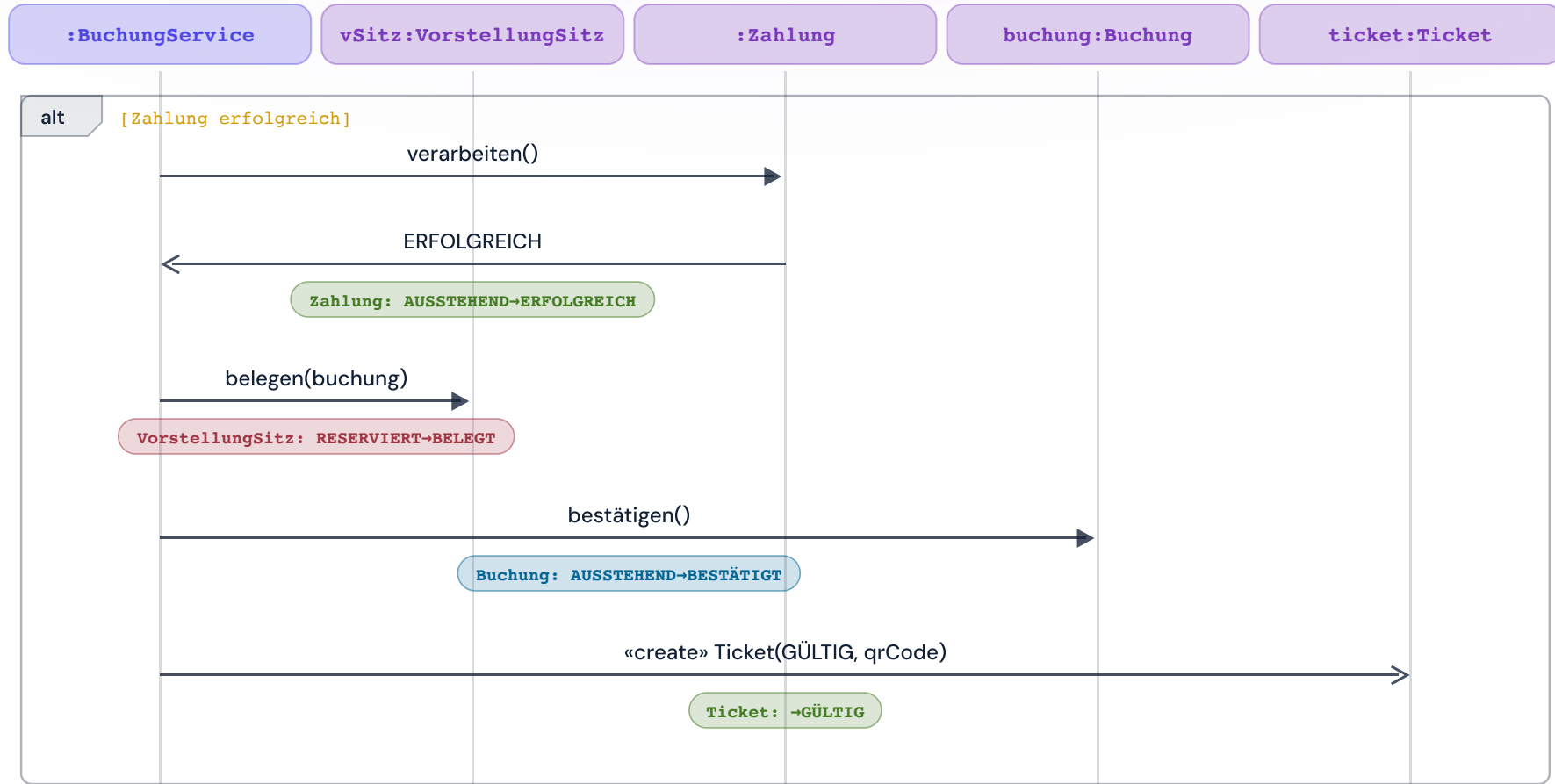


Beim Checkout ruft der BuchungService reservieren() am VorstellungSitz auf. Der Sitz wird serverseitig RESERVIERT und ist für parallele Käufer blockiert. Das Badge zeigt den Statuswechsel.

Kommt reservieren() zu spät, antwortet der Sitz mit false — der Kunde landet in der Sitzwahl, nie in einer Doppelbuchung.

SEQUENZDIAGRAMME

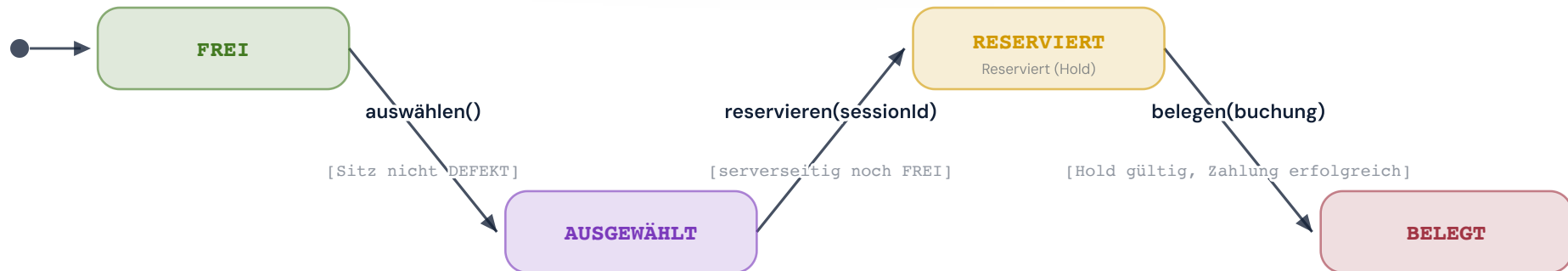
Die Zahlung entscheidet



Erst wenn die Zahlung ERFOLGREICH ist, wird der Sitz BELEGT, die Buchung BESTÄTIGT und das QR-Ticket erzeugt.

ZUSTANDSDIAGRAMME

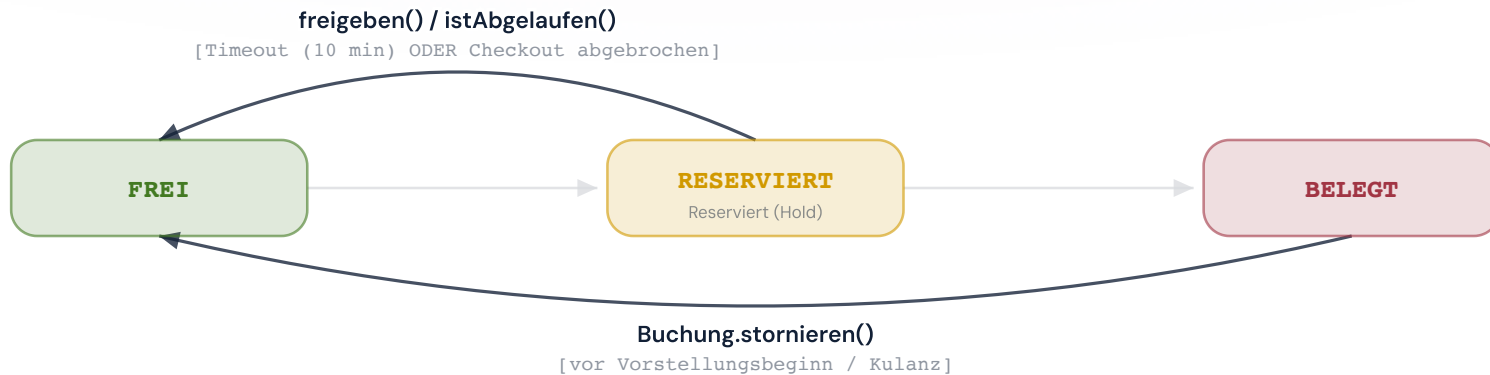
Der wichtigste Automat: der Sitz



Ein Zustandsautomat beschreibt genau ein Objekt über die Zeit. Der wichtigste ist der Sitz je Vorstellung: auswählen ist nur lokal, reservieren() setzt den serverseitigen Hold, und erst belegen() nach erfolgreicher Zahlung macht den Platz endgültig zu.

ZUSTANDSDIAGRAMME

Jeder Hold endet von selbst



Läuft der 10-Minuten-Hold ab oder bricht der Kunde ab, wird der Sitz automatisch wieder FREI. Ein Storno gibt auch belegte Plätze zurück, und DEFEKT sperrt einen Sitz bei Bedarf systemweit. Kein Sitz bleibt für immer blockiert.

Jeder Automat ist wertgleich mit seinem Status-Enum im Klassendiagramm — Struktur und Verhalten widersprechen sich nie.

ZUSTANDSDIAGRAMME

Die weiteren Automaten in Kürze

Ticket-Lebenszyklus

4 Zustände

Ein Anfang, drei Enden — eingelöst, storniert oder abgelaufen.

Buchungs-Lebenszyklus

4 Zustände

Hält alles zusammen — bestätigt erst mit erfolgreicher Zahlung.

Zahlungs-Lebenszyklus

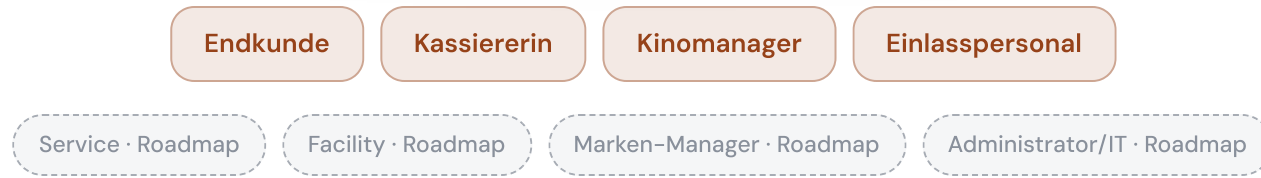
4 Zustände

Entscheidet den Rest — und erst ein Storno macht daraus ERSTATTET.

Drei weitere Objekte haben ihren eigenen Lebenszyklus — nach demselben Muster, gekoppelt über die Buchung.

PROTOTYP

Vom Modell zur App



4 von 8 Rollen und 41 von 82 UML-Klassen sind gebaut.

Der Prototyp setzt den Kern des Modells klickbar um; die übrigen Rollen sind bewusst als Roadmap modelliert, nicht vergessen. Das Modell beschreibt das ganze Produkt — der Prototyp seinen Kern.

Modell 2 Prototyp: Was noch nicht gebaut ist, ist im Klassendiagramm bereits verortet.

PROTOTYP

Live-Demo

 [LIVE-DEMO]

1

Endkunde

buchen

2

Kasse

verkaufen

3

Manager

steuern

4

Einlass

scannen

Buchen im Sitzplan, verkaufen an der Kasse, steuern im Manager-Cockpit, scannen am Einlass — die echte App läuft direkt im Browser, mit Mock-API.

START

PERSONAS

STORIES

STORY MAP

KLASSEN

SEQUENZEN

ZUSTANDE

PROTOTYP

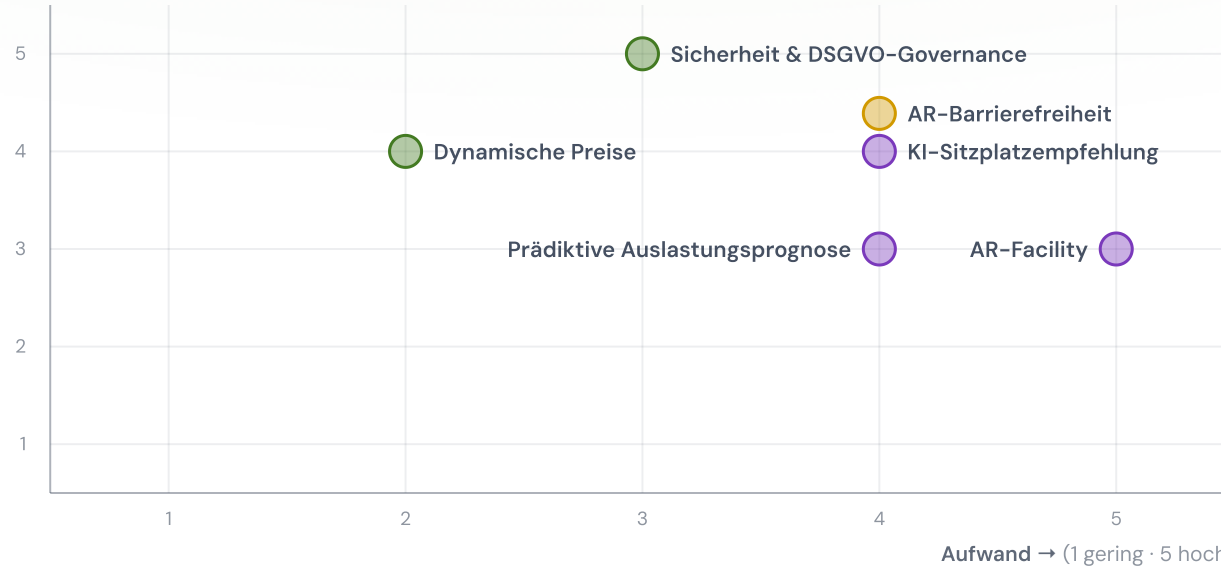
INNOVATION

FAZIT

INNOVATION

Sechs Ideen, nach Wirkung und Aufwand

Impact ↑ (1 gering · 5 hoch)



● im Rahmen machbar

● teilweise machbar

● Konzept / Vision

Recherchierte Ideen für die Zeit nach dem MVP, eingeordnet nach Impact und Aufwand. Manche sind im Rahmen machbar, andere bewusst Konzept.

INNOVATION

Nah am MVP

Dynamische Preise

im Rahmen machbar

Ticketpreise passen sich regelbasiert an die Auslastung an, um Nachfragespitzen besser zu nutzen.

Sicherheit & DSGVO-Governance

im Rahmen machbar

Least-Privilege-Rollen, lückenloses Audit-Log, erzwungene 2FA sowie DSGVO-Auskunft und -Löschung.

AR-Barrierefreiheit

teilweise machbar

Zu einer Vorstellung lässt sich eine Untertitel- oder Audiodeskriptions-Brille dazubuchen.

Drei Ideen liegen nah am bestehenden System — im Projektrahmen umsetzbar oder in Teilen machbar.

INNOVATION

Drei Blicke nach vorn

KI-Sitzplatzempfehlung

Konzept / Vision

Die KI schlägt den besten freien Platz passend zu Präferenzen vor – kein langes Suchen im Sitzplan.

AR-Facility: Defekt melden & Sitz sperren

Konzept / Vision

Haustechnik meldet einen defekten Sitz per AR-Brille mit Foto und Sprachnotiz; sofort entsteht ein Wartungsticket und der Sitz wird gesperrt.

Prädiktive Auslastungsprognose

Konzept / Vision

Die erwartete Auslastung kommender Vorstellungen wird prognostiziert, um Programm und Personal vorausschauend zu planen.

Drei weitere Ideen sind bewusst als Konzept gedacht — recherchiert und eingeordnet, aber noch nicht eingeplant.

ABSCHLUSS

Drei Kernsätze

1**Vom Bedarf zum Modell**

Jede Anforderung ist als Persona, Story und Release verortet — nichts hängt in der Luft.

2**Ein Modell ohne Widersprüche**

Klassen, Sequenzen und Zustände greifen ineinander; die Status-Enums halten sie wertgleich.

3**Nicht nur beschrieben — gebaut**

Der MVP läuft als klickbarer Prototyp; alles Weitere ist als Roadmap modelliert.

Vielen Dank — Fragen und Diskussion.